

SI MATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA14 COURSE NAME: COMPILER DESIGN

COURSE OUTCOME COVERED

C05: Develop code optimization and code generation using DAG representation with data flow analysis to produce optimized target code.

Assessment Tool Weightage: 40%

Duration: 60 Mins

On Class

QUESTIONS: 5

Total Marks: 50

Annexure A

SIMULATION TASK

Code of Conduct:

I V G ABHIRAM - 192524097 certify that this submission is my original work and that I have adhered to the specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: V G ABHIRAM

Q. No	Conceptual Understanding (10)	Model Design (25)	Tool Usage (10)	Analysis of Results (10)	Documentation (5)	Total Score (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
OVERALL RUBRICS VALUE						
	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 20	/ 25	/ 20	/ 20	/ 15	/ 100

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Conceptual Understanding	20%	Demonstrates strong understanding of underlying concepts in the simulation	Shows good understanding with minor gaps	Partial understanding with some misconceptions	Lacks understanding of key concepts
Model Design	25%	Develops accurate, well-structured simulation model independently	Develops correct model with minor errors	Model is incomplete or requires guidance	Unable to develop a proper model
Tool Usage	20%	Uses simulation tools effectively with correct setup and execution	Uses tools with minor errors or guidance	Limited ability to use tools properly	Unable to use simulation tools
Analysis of Results	20%	Provides clear, logical analysis and meaningful interpretation of results	Analysis is reasonable with minor gaps	Superficial analysis with limited interpretation	Unable to analyze or interpret results
Documentation	15%	Presents results clearly with proper documentation and structured reporting	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation presentation

Annexure A

SIMULATION TASK

Q1. Instruction Selection, Register Allocation and Target Code Generation

Consider the following intermediate representation of an arithmetic expression

$t1 = b + c$

$t2 = t1 * d$

$t3 = e - f$

$t4 = t2 + t3$

$a = t4$

i. Instruction Selection

Instruction selection is the process of converting intermediate code into suitable target machine instructions. For the above intermediate representation, addition, multiplication, subtraction and move instructions are selected.

ADD R1, b, c

MUL R2, R1, d

SUB R3, e, f

ADD R4, R2, R3

MOV a, R4

The first instruction adds b and c and stores the result in R1. The second instruction multiplies the value in R1 with d and stores the result in R2. Similarly e-f is calculated in R3, and finally the two intermediate results are added.

ii. Register Allocation

Registers are allocated to store intermediate values so that unnecessary memory access can be avoided.

R1 stores t1

R2 stores t2

R3 stores t3

R4 stores t4

Assume the input values are b = 2, c = 3, d = 4, e = 10, and f = 6.

iii. Target Instruction Sequence

The complete target instruction sequence is:

ADD R1, b, c

MUL R2, R1, d

SUB R3, e, f

ADD R4, R2, R3

MOV a, R4

iv. Register Contents After Each Instruction

Initially all registers are empty. After the ADD R1, b, c, $R1 = 2 + 3 = 5$. After multiplication, $R2 = 5 \times 4 = 20$. The subtraction produces $R3 = 10 - 6 = 4$. Finally $R4 = 20 + 4 = 24$, which is moved to a.

Therefore, the final value is:

a = 24

The register contents change as follows during execution

ADD R1, b, c → $R1 = 5$

MUL R2, R1, d → $R2 = 20$

SUB R3, e, f → $R3 = 4$

ADD R4, R2, R3 → $R4 = 24$

MOV a, R4 → a = 24

v. Efficiency of Generated Code

The generated code is efficient because each arithmetic operation is directly translated into a target instruction. Intermediate values are stored in the registers, reducing memory access. Only five target instructions are required for the complete

expressed properly, register allocation also reduces execution time and improves processor utilization.

Conclusion Efficient instruction selection and register allocation produce compact target code with fewer memory operations and better execution performance.

Q2. Runtime Stack and Activation Records

Consider a program containing nested procedure calls:

```
main()
```

↓

```
P(10)
```

↓

```
Q(20)
```

Procedure P has parameter a and local variable x. Procedure Q has parameter b and local variable y.

i. Runtime Stack Creation

The runtime stack is used to store information required during procedure execution. When main() begins, its activation record is created on the stack. When P(10) is called, another activation record is pushed above main(). When Q(20) is called from P, another activation record is created on top.

The stack during the call to Q is:

```
| Q : parameter b=20, local y |
|-----|
```

```

| P : parameter a=10, local x |
|-----|
| main activation record      |

```

The most recently called procedure is always at the top of the stack.

ii. Construction of Activation Records

An activation record is a block of memory created for every procedure invocation. It contains the information required to execute that procedure and return to its caller.

A typical activation record contains:

- Parameters passed to the procedure
- Local variables
- Return address
- Control link
- Access link, when required
- Temporary values
- Saved registers

For procedure P, the activation record stores parameter a = 10 and local variable x. For procedure Q, it stores parameter b = 20 and local variable y.

iii. Storage Allocation

When P(10) is called, storage is allocated for its parameter and local variables. When Q(20) is called, another area is allocated for its parameter and local variable. Thus, each procedure gets its own separate storage. Changes made to the local variables of one procedure do not overwrite the local variables of another active procedure.

iv. Stack Changes During Calls and Returns

Initially:

| main |

After calling P(10):

| P : a=10, x |

|-----|

| main |

After P calls Q(20):

| Q : b=20, y |

|-----|

| P : a=10, x |

|-----|

| main |

When Q completes, its activation record is removed:

| P : a=10, x |

|-----|

| main |

When P completes, its activation record is also removed:

| main |

Finally, when main finishes, the runtime stack becomes empty.

v. Role of Runtime Storage Management

Runtime storage management is essential during code generation because the compiler must generate code that correctly handles procedural calls, parameters, local variables, temporary values and return addresses. The runtime stack provides storage for active procedures and follows the Last-In-First-Out (LIFO) principle.

It also allows recursive and nested procedures to execute correctly. When a procedure is called, its activation records are created, and when it returns, the activation records are removed and its storage is released.

Conclusion

Runtime storage management provides an organized way of allocating and releasing memory during program execution and ensures that procedural calls and returns work correctly.

Q8. Next-Use Information and Register Allocation

Consider the following intermediate code:

1. $a = b + c$
2. $d = a + e$
3. $f = d * g$
4. $h = f + i$

i. Construction of Next-Use Table

Next-use information tells the compiler when a variable will be used again. It helps determine whether the value of a variable should remain in a register or whether its register can be reused.

For the above code:

In statement 1, a is used again in statement 2.

In statement 2, d is used again in statement 3.

In statement 3, f is used again in statement 4.

After their final use, these variables become inactive.

Therefore, the important next-use information is:

$a \rightarrow$ next used at statement 2

$d \rightarrow$ next used at statement 3

$f \rightarrow$ next used at statement 4

Variables such as b , c , e , g , and i have no further use after the corresponding

statement.

ii. Register Allocation

Assume three registers: R1, R2, and R3.

The compiler can allocate registers as follows:

R1 → a

R2 → d

R3 → f

The first instruction calculates a and stores it in R1:

ADD R1, b, c

The second instruction uses R1 to calculate d:

ADD R2, R1, e

After this instruction, a is no longer needed, so R1 can be reused.

The next instruction calculates f:

MUL R1, R2, g

Here R1 is reused for f.

iii. Releasing Inactive Registers

After statement 2, a has no future use. Therefore, its register can be released.

After statement 3, d has no future use, so its register can also be reused.

After statement 4, f becomes inactive and its register can be released.

This avoids keeping unnecessary values in registers.

iv. Final Target Instruction Sequence

ADD R1, b, c

ADD R2, R1, e

MUL R1, R2, g

```
ADD R2, R1,
```

```
MOV h, R2
```

Here the same registers are reused whenever their previous contents are no longer required.

v. How Next-Use Information Improves Register Utilization

Next-use information helps the compiler decide which values should remain in registers and which registers can be reused. If a variable will not be used again, the register containing its value can immediately be released. If all registers are occupied, the compiler can choose a variable with no future use or a variable whose next use is farthest away for replacement.

This reduces register spilling and unnecessary memory operations. It also improves execution speed because registers are much faster to access than main memory.

Conclusion: Next-use information enables efficient register allocation by identifying inactive values and allowing registers to be reused effectively.

Q4. Loop-Invariant Code Motion

Consider the following intermediate code:

```
while(i < n)
{
    t1 = a * b;
    t2 = t1 + i;
    x[i] = t2;
    i = i + 1;
}
```

i. Identifying Repeated Computations

The statement $t1 = a * b$ is executed every time the loop runs. However, the values of a and b do not change inside the loop.

Therefore, $a * b$ always produces the same result. This makes it a loop-invariant computation.

The other computation $t2 = t1 + i$ depends on i , which changes in every iteration.

Therefore, it cannot be moved outside the loop.

ii. Moving the Loop-Invariant Computation

The loop-invariant statement can be calculated once before the loop:

```
t1 = a * b;
```

```
while(i < n)
{
    t2 = t1 + i;
    x[i] = t2;
    i = i + 1;
}
```

Instead of calculating $a * b$ repeatedly, the compiler calculates it only once and reuses the result.

iii. Optimized Intermediate Code

The optimized code is:

```
t1 = a * b
```

```
L1:
```

```
    if i >= n goto L2
```

```

t2 = t1 + i
x[i] = t2
i = i + 1
goto L1

```

L2:

This produces the same output as the original program but performs fewer arithmetic operations.

iv. Comparison of Execution Cost

Assume the loop executes 100 times.

In the original code $a * b$ is calculated 100 times. The $t1 + i$ operation is also calculated 100 times. Therefore, there are 200 arithmetic operations.

After optimization $a * b$ is calculated only once while $t1 + i$ is still calculated 100 times.

Therefore, there are only 101 arithmetic operations.

Original code $\rightarrow 100 + 100 = 200$ operations

Optimized code $\rightarrow 1 + 100 = 101$ operations

Thus, the optimized version reduces the number of arithmetic operations by 99.

v. Why Loop Optimization Improves Performance

Loops can execute hundreds, thousands, or even millions of times. Any unnecessary computation inside a loop is therefore repeated many times and increases execution time.

Loop-invariant motion removes computations that produce the same result in every iteration.

It reduces CPU workload, decreases execution time, reduces unnecessary instructions, and improves overall program performance.

Conclusion: Moving loop-invariant computations outside the loop is an effective

optimization technique, especially for loops with a large number of iterations.

Q5. DAG-Based Common Subexpression Elimination

Consider the following basic block

$t1 = a + b$

$t2 = c * d$

$t3 = a + b$

$t4 = t1 * t2$

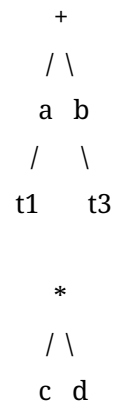
$t5 = t3 + t2$

i. Construction of DAG

A Directed Acyclic Graph (DAG) is used to represent the computations inside a basic block. In the DAG, operands are represented as leaf nodes and operators are represented as internal nodes.

For the above code, $a + b$ occurs in both $t1$ and $t3$. Therefore, only one $+$ node is required for this expression.

The important DAG structure is:



\ /
t2

The same $a+b$ node is shared by t1 and t3. This indicates that the two expressions produce the same value.

ii. Identifying Common Subexpressions

The original code contains:

t1 = a + b

t3 = a + b

The expression $a+b$ is a common subexpression because it is calculated more than once and its operands have not changed between the two computations.

Therefore, calculating it again is unnecessary.

iii. Eliminating Redundant Computation

The first calculation of $a+b$ is retained and its result is reused for t3.

Original:

t1 = a + b

t2 = c * d

t3 = a + b

t4 = t1 * t2

t5 = t3 + t2

Optimized:

t1 = a + b

t2 = c * d

t4 = t1 * t2

t5 = t1 + t2

The redundant statement $t3 = a + b$ is removed because t1 already contains the result.

iv. Optimized Intermediate Instructions

The final optimized intermediate code is:

$t1 = a + b$

$t2 = c * d$

$t4 = t1 * t2$

$t5 = t1 + t2$

The value calculated by $t1$ is reused when calculating $t5$.

v. Comparison of Original and Optimized Code

The original code contains four arithmetic computations:

$a+b$

$c*d$

$a+b$

$t1*t2$

$t3+t2$

Actually, this is five arithmetic statements repeated twice.

After optimization, there are only four arithmetic statements:

$a+b$

$c*d$

$t1*t2$

$t1+t2$

Thus, one redundant computation is eliminated.

The optimized code requires fewer instructions, performs less work, and uses the already calculated result instead of recalculating the same expression. This improves execution efficiency and reduces the computational cost of the basic block.

Conclusion:

DAG-based optimization detects common subexpressions and eliminates redundant

calculations, producing shorter and more efficient intermediate code.